Reinforcement Learning-Based Computer Numerical Control Motion Planning with Adaptive Preview and Jerk-Constrained Action Projection

Author Name[a]

[a] *Organization Name, Address Line, City, Postcode, Country*

Abstract

Computer numerical control (CNC) machining requires motion planning methods that smooth mixed tool paths within a prescribed contour-tolerance band, maintain feed efficiency, and satisfy limits on velocity, acceleration, and jerk. This paper presents a structured reinforcement learning (RL) decision framework for tolerance-band CNC motion planning. The framework uses Proximal Policy Optimization (PPO) to learn steering, feedrate, and preview-distance actions, allowing the policy to choose the geometric perception scale needed for straight, corner-entry, corner-passing, and corner-exit phases. It also includes an interpretable kinematic constraint module (KCM), which acts as a jerk-constrained action-projection layer and maps unconstrained policy outputs to interpolation commands that satisfy limits on linear and angular velocity, acceleration, and jerk. Experiments on square, circular, and butterfly paths show that the proposed method produces smoothed trajectories within the tolerance band and avoids the jerk violations found in the neural network controller (NNC) baseline, improving the dynamic feasibility of CNC motion planning under complex geometric transitions.

Keywords: computer numerical control, reinforcement learning, learnable preview distance, jerk-constrained action projection

1. Introduction

In high-speed, high-precision machining, CAM-generated tool paths are often represented as discrete line and circular-arc segments with nonuniform lengths [1, 2]. Discontinuities in tangent direction and curvature at program-segment junctions can degrade geometric continuity and kinematic feasibility [3]. A machine tool therefore often has to decelerate near corners to maintain contour accuracy, while direct high-speed traversal may lead to excessive acceleration, jerk, and contour error. The main task of CNC motion planning is to convert a discrete CAM-generated tool path into an executable interpolation trajectory that satisfies the contour tolerance and the limits on velocity, acceleration, and jerk. In this sense, motion planning directly affects both machining quality and machining efficiency [1, 4].

Traditional CNC motion planning is commonly organized as a sequential process, where trajectory smoothing is carried out before feedrate planning [1, 3]. Trajectory smoothing improves the geometric continuity of discrete program segments. Local smoothing methods construct smooth transition curves between adjacent segments [3, 5]. For example, Li et al. [6] proposed a motion-overlap method for mixed line–arc tool paths, where overlapping transitions between adjacent motion segments improve connection con-

tinuity. Zhao et al. [7] used curvature-continuous B-spline transitions for real-time look-ahead interpolation of short line segments, thereby improving the geometric continuity of corner transitions. Global methods instead reconstruct the discrete path over the entire domain. Song et al. [8] combined control-point allocation with FIR filtering for global smoothing of continuous short-line tool paths. Sun et al. [9] studied a smooth interpolation method for five-axis machining paths to improve interpolation efficiency on complex paths. Feedrate planning then generates feedrate profiles along a prescribed geometric trajectory [1, 4]. Lee et al. [10] examined feedrate scheduling in NURBS interpolation. Erkorkmaz and Altintas [11] improved dynamic smoothness in high-speed CNC machining through jerk-limited trajectory generation and quintic spline interpolation. Song et al. [12] developed an adaptive feedrate-planning method for long five-axis spline paths using a look-ahead window and axis-drive constraints.

Existing studies show that, when trajectory smoothing is performed before feedrate planning, the feedrate-planning result is bounded by the generated geometric trajectory [13]. Under jerk constraints and nonstationary boundary conditions, subsequent feedrate planning mainly improves dynamic feasibility along a prescribed trajectory [14, 15]. Once the geometric trajectory is fixed, feedrate planning can mainly adjust the motion speed along that trajectory, with limited ability to reshape the executed path in corner regions. Studies on look-ahead windows and dynamic look-ahead control also show that upcoming geometric information affects both feedrate planning and the local geometric response. A fixed preview range provides limited adaptability across different path stages, including straight-line feed, corner approach, corner traversal, and corner exit [12, 16]. These limitations call for a unified closed-loop motion-planning framework in which the controller generates both trajectories within the tolerance band and feedrates that satisfy kinematic constraints.

Recent work has therefore considered learning-based policies that directly generate interpolation control actions from the current tool path, upcoming geometric information, and motion state. Li et al. were among the first to apply reinforcement learning to CNC trajectory smoothing and proposed the neural-network numerical control (NNC) direct trajectory smoothing method [2]. This method formulates trajectory smoothing as an action-decision problem. At each interpolation cycle, a neural network outputs servo control commands from the current tool path and operating state. The action consists of angle-change and step-length-change components, which are then converted into a position command. The reward is mainly defined by the contour error from the current point to the G-code path. The reported experiments show that NNC achieves higher machining efficiency than local smoothing methods and better contour accuracy than global smoothing methods. Beyond NNC, Samsonov et al. [17] studied CNC manufacturing decision problems using deep representation learning and reinforcement learning. Alizadeh Kolagar et al. [18] combined deep reinforcement learning with a jerk-bounded trajectory generator to improve kinematic feasibility in learning-based control. In safe learning-based control, OptLayer proposed by Pham et al. [19], the shielding approach of Alshiekh et al. [20], control-barrier-function-based safety control developed by Cheng et al. [21], and the predictive safety filter introduced by Wabersich and Zeilinger [22] show that learned actions often need projection or filtering mechanisms to satisfy explicit constraints.

The NNC method described above leaves two issues unresolved [2]. The neural network action is converted directly into an interpolation position or a servo command. Without explicit treatment of velocity, acceleration, and jerk constraints, or a projection

step at the execution layer, the resulting command may exceed the kinematic limits of the machine tool. The reward design is also centered mainly on contour error, while path progress, direction consistency, motion smoothness, and the stage-dependent behavior near corners receive limited attention. This makes it difficult for the policy to use the tolerance band to choose smooth transition trajectories at sharp corners or in regions where the curvature changes rapidly.

A reinforcement learning-based CNC motion-planning method with a learnable look-ahead distance and jerk-constrained action projection is proposed. This framework is referred to as J-NNC (Jerk-constrained Neural Numerical Controller) to distinguish it from direct neural control methods such as NNC. The method formulates motion planning for mixed tool paths as a constrained sequential decision-making problem within the tolerance band. In this formulation, the policy outputs steering, feedrate, and look-ahead-distance adjustment actions in a closed loop. The state representation includes corner information and multi-scale forward geometric information, and the reward considers path progress, contour error, direction consistency, and motion smoothness. A kinematic constraint module (KCM) maps the unconstrained control variables to interpolation control variables that satisfy the velocity, acceleration, and jerk constraints. Fig. 1 compares the conventional serial motion-planning framework with the closed-loop J-NNC motion-planning framework.

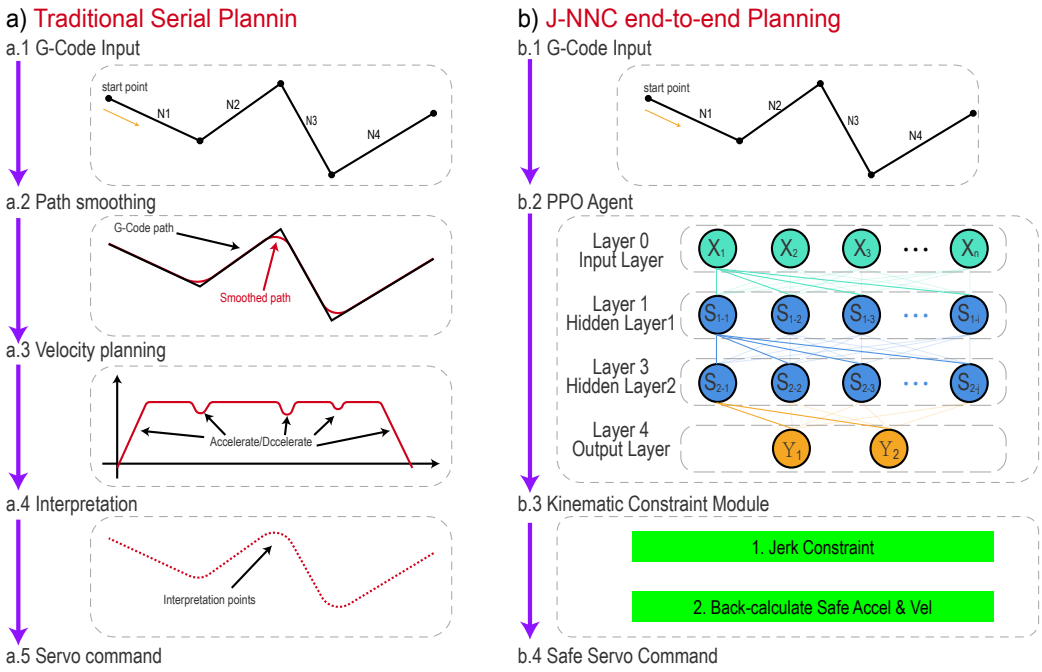

Fig. 1: Comparison between conventional serial motion planning and the J-NNC motion-planning framework. J-NNC denotes the Jerk-constrained Neural Numerical Controller, i.e., a neural-network-based CNC motion planner with jerk-constrained action projection.

The method makes three main contributions. It uses the real-time kinematic state, contour error, corner information, and multi-scale forward geometric information as the policy input, and treats the look-ahead distance as a learnable action. This allows the policy to adjust its perception scale across different geometric stages and to generate smooth local transition trajectories within the tolerance band. The method also introduces a KCM between the policy output and the execution layer. This module converts

the steering and feedrate commands generated by the policy into executable control variables that satisfy the prescribed bounds on velocity, acceleration, and jerk, which improves the kinematic feasibility of the learned policy output. Comparative and ablation experiments on the square, circle, and butterfly paths show that J-NNC completes all test cases, with a final path progress of 1.000 and a maximum relative violation of linear jerk of 0. Compared with the NNC baseline and the ablated configuration without KCM, the proposed method avoids jerk-limit violations while preserving path progress.

The contributions of this paper are summarized as follows:

(1) The structured state encodes tracking, kinematic, corner, and preview-geometry information, while the steering, feedrate, and preview-distance actions enable the agent to perceive upcoming corners and select smoother trajectories within the tolerance band.

(2) The KCM maps raw steering and feedrate outputs into executable interpolation commands while enforcing velocity, acceleration, and jerk limits at the execution layer.

(3) Experiments on square, circular, and butterfly-shaped paths show that J-NNC preserves full path progress and eliminates the linear-jerk violations observed in the NNC baseline and the no-KCM ablation.

The remainder of this paper is organized as follows. Section 2 defines the problem and models the constraints. Section 3 presents the proposed method. Section 4 reports the experimental settings, comparative results, and ablation analysis. Section 5 concludes the paper.

2. Problem Formulation

This section defines the motion-planning problem considered in this study, together with the geometric and execution constraints used in the proposed method. These definitions provide the basis for the method described in the following sections.

2.1. Tool Path and Geometric Feasible Region

Consider a discrete sequence of cutter-location points generated by a CAM system,

$$P_{\text{raw}} = p_1, p_2, \ldots, p_N, \tag{1}$$

where $p_i \in \mathbb{R}^2$ denotes a reference point in the interpolation plane. Let ε denote the prescribed contour tolerance. The reference path is offset by $\varepsilon/2$ on both sides to define the geometric feasible region in which the policy may locally deviate from the reference path:

$$\Omega = x \mid d(x, P_{\text{raw}}) \leq \varepsilon/2, \tag{2}$$

where $d(x, P_{\text{raw}})$ denotes the minimum distance from point x to the reference path. This region allows local redistribution of the trajectory within the tolerance band, which helps smooth corner traversal while preserving contour accuracy.

For closed-loop decision making, the reference path is parameterized by arc length. At the current projected arc length ℓ_t, local geometric quantities are extracted, including the normal error, tangential direction, distance to the next corner, and turning angle of the next corner. These quantities are used in the state representation, reward weighting, and corner-mode identification.

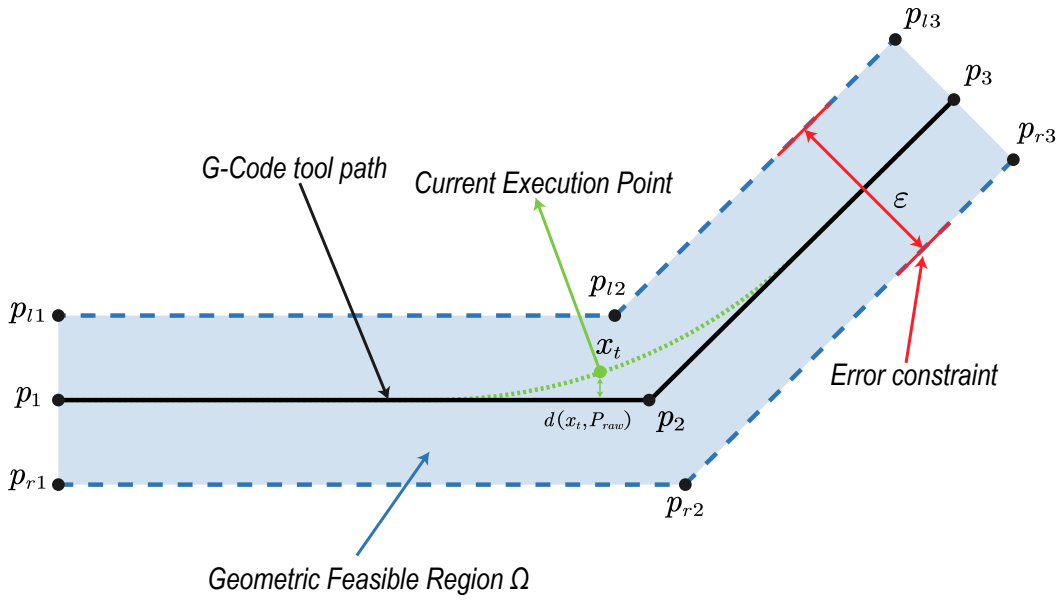

Fig. 2: Geometric feasible region formed by the reference path and the contour tolerance band.

2.2. Constrained Action Decision Formulation

At each interpolation period Δt, the policy receives the current state s_t and outputs a normalized action a_t^{π}. The kinematic constraint module (KCM) projects this action into an executable control command, which is then applied to the environment. The environment updates the position, orientation, and kinematic state, and returns the immediate reward r_t. The task is therefore written as a constrained sequential decision-making problem:

$$\max_{\pi} \ \mathbb{E}\pi \left[\sum t = 0^{T} \gamma^{t} r_t \right], \tag{3}$$

subject to the geometric constraint

$$x_t \in \Omega, \tag{4}$$

and the execution-side kinematic constraints

$$|v_t| \le V_{\max}, \quad |a_t| \le A_{\max}, \quad |j_t| \le J_{\max}, \tag{5}$$

$$|\omega_t| \le \Omega_{\max}, \quad |\dot{\omega} t| \le A\max^{\omega}, \quad |\ddot{\omega} t| \le J\max^{\omega}. \tag{6}$$

Here, v_t, a_t, and j_t denote the linear velocity, linear acceleration, and linear jerk, respectively, whereas ω_t, $\dot{\omega}_t$, and $\ddot{\omega}_t$ denote the angular velocity, angular acceleration, and angular jerk, respectively.

3. Methodology

3.1. Overall Framework

Motion planning for hybrid tool paths is formulated as a closed-loop decision process in a reinforcement learning setting. The policy network acts as the agent, and the tool-path simulation system, which accounts for contour tolerance and kinematic constraints, acts as the environment. At each interpolation cycle, the environment builds the state s_t from the current execution position, kinematic state, local corner information, and look-ahead geometry. Given s_t, the policy π_θ produces an action a_t that gives the steering,

feedrate, and look-ahead-distance adjustment commands for the current cycle. The action is corrected by the KCM and then applied to the environment. The environment moves to the next state s_{t+1} and returns a reward r_t based on path progress, contour error, and motion smoothness. During training, reward feedback updates the policy parameters, enabling the policy to learn a control law for trajectory smoothing, feedrate planning, and kinematic-constraint satisfaction within the tolerance band.

Fig. 3 shows the overall framework of the proposed method. Given a reference path and a contour tolerance, the environment carries out path projection, corner scanning, and look-ahead geometry extraction to form a structured state vector at the current time step. The policy network outputs a normalized action. The steering and feedrate components are mapped to the pre-projection motion control variables $(\omega_t^{pre}, v_t^{pre})$, and the look-ahead component is mapped to the current look-ahead distance L_t. The KCM then uses the velocity, acceleration, and jerk states from the previous time step to project $(\omega_t^{pre}, v_t^{pre})$ onto the feasible kinematic set. This projection gives the executable control variables (ω_t, v_t), which satisfy the velocity, acceleration, and jerk limits. The execution result updates the current position, kinematic quantities, corner phase, and look-ahead observations. During training, the same result also enters the policy update through the reward signal. The whole procedure forms a closed reinforcement learning loop for motion planning.

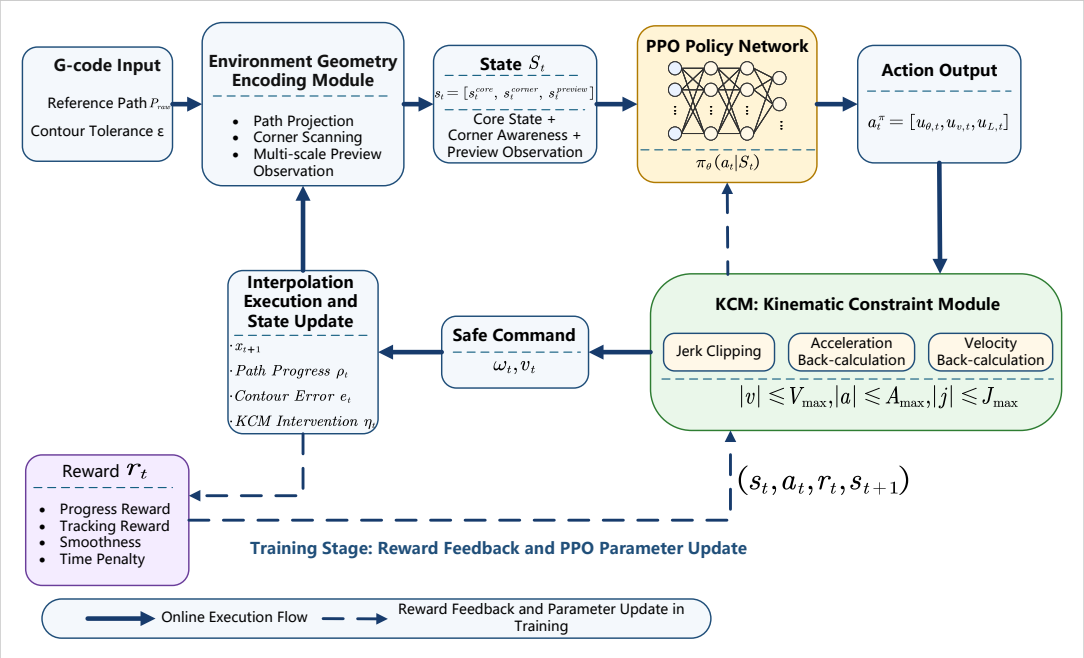

Fig. 3: Overall closed-loop framework of the proposed method.

3.2. State Design

A hierarchical structured state representation is used:

$$s_t = [s_t^{core},\ s_t^{corner},\ s_t^{preview}]. \tag{7}$$

Here, s_t^{core} describes the local tracking and kinematic state at the current time step, s_t^{corner} describes the relation between the current path segment and the next corner, and $s_t^{preview}$ records the geometric trend over several preview distances ahead.

The core state is an eight-dimensional vector:

$$s_t^{core} = [\bar{e}_t,\ \bar{e}_{n,t},\ \bar{\tau}_t,\ \bar{v}_t,\ \bar{a}_t,\ \bar{\omega}_t,\ \rho_t,\ \bar{d}_{c,t}], \tag{8}$$

where $\bar{e}_t$ is the normalized contour error, $\bar{e}_{n,t}$ is the signed normal error, and $\bar{\tau}_t$ is the normalized angular error between the current heading and the local tangent direction. The terms $\bar{v}_t$, $\bar{a}_t$, and $\bar{\omega}_t$ are the normalized linear velocity, linear acceleration, and angular velocity, respectively. In addition, $\rho_t \in [0, 1]$ gives the overall path progress, and $\bar{d}_{c,t}$ is the normalized distance to the next corner.

The corner-aware state is a four-dimensional feature vector:

$$s_t^{corner} = [\bar{\phi}_t,\ \sigma_t,\ m_t,\ \sigma_t \bar{e}_{n,t}], \tag{9}$$

where $\bar{\phi}_t$ is the normalized turning angle of the next corner, $\sigma_t \in -1, 0, 1$ is the turning sign of the corner, and $m_t \in 0, 1$ indicates whether the current position is within the corner-influence interval. The last term, $\sigma_t \bar{e}_{n,t}$, encodes the signed positional relation to the inner or outer side of the turn.

For preview observation, several preview scales $\lambda_k k = 1^{N_L}$ are defined along the arc length of the reference path. Let ℓ_t be the arc-length coordinate of the projection of the current execution point onto the reference path. The forward observation point for the k-th preview scale is

$$q_t^{(k)} = P\mathrm{raw}(\ell_t + \lambda_k L_t), \qquad k = 1, \ldots, N_L, \tag{10}$$

where λ_k is the scale coefficient and L_t is the base preview distance used in the current cycle. In this formulation, the near-, medium-, and far-range preview points are determined along the arc length of the reference path, rather than from ray intersections along the current execution direction. For each scale, the angular variation of the forward observation point relative to the current tangent direction and the corresponding distance ratio are extracted:

$$s_t^{preview} = \left[\Delta\theta_t^{(1)},\ \bar{d}_t^{(1)},\ \ldots,\ \Delta\theta_t^{(N_L)},\ \bar{d}_t^{(N_L)}\right]. \tag{11}$$

Here, $\Delta\theta_t^{(k)}$ describes the directional change at the k-th preview scale, and $\bar{d}_t^{(k)}$ is the normalized distance from the current position to the corresponding preview point. The main implementation uses three preview scales.

3.3. Action Space and Kinematic Constraint Module

At each control cycle, the policy network outputs a normalized action:

$$a_t^{\pi} = [u_{\theta,t}, u_{v,t}, u_{L,t}], \tag{12}$$

where $u_{\theta,t} \in [-1, 1]$ is the normalized steering component, $u_{v,t} \in [0, 1]$ is the normalized feedrate component, and $u_{L,t} \in [0, 1]$ is the preview-distance adjustment component. Given the admissible preview-distance range $[L_{\min}, L_{\max}]$, the physical preview distance is mapped as

$$L_t = L_{\min} + u_{L,t}(L_{\max} - L_{\min}). \tag{13}$$

The policy output is first converted to pre-projection motion control variables in physical units:

$$\omega_t^{pre} = u_{\theta,t}\Omega_{\max}, \qquad v_t^{pre} = u_{v,t}V_{\max}. \tag{14}$$

The preview distance L_t is used to compute the local reference direction and preview geometry. The KCM then projects $(\omega_t^{pre}, v_t^{pre})$ onto the kinematically feasible set according to the kinematic state at the previous time step. For the linear-velocity channel, let v_{t-1} and a_{t-1} be the linear velocity and linear acceleration in the previous cycle. The pre-projection acceleration is

$$a_t^{pre} = \frac{v_t^{pre} - v_{t-1}}{\Delta t}, \tag{15}$$

and the corresponding pre-projection jerk is

$$j_t^{pre} = \frac{a_t^{pre} - a_{t-1}}{\Delta t}. \tag{16}$$

The KCM clips this jerk by the jerk bound:

$$j_t = \mathrm{clip}(j_t^{pre}, -J_{\max}, J_{\max}), \tag{17}$$

and then reconstructs the realizable acceleration and velocity for the current cycle:

$$a_t = \mathrm{clip}(a_{t-1} + j_t \Delta t, -A_{\max}, A_{\max}), \tag{18}$$

$$v_t = \mathrm{clip}\left(v_{t-1} + a_{t-1}\Delta t + \frac{1}{2} j_t \Delta t^2, 0, V_{\max}\right). \tag{19}$$

The angular-velocity channel uses the same projection procedure. The resulting (ω_t, v_t) is then applied to the environment as the executable control input. The resulting online projection procedure is summarized in Algorithm 1.

Algorithm 1 KCM-constrained action projection

Require: Policy output $a_t^{\pi} = [u_{\theta,t}, u_{v,t}, u_{L,t}]$; previous-step kinematic state $(v_{t-1}, a_{t-1}, \omega_{t-1}, \alpha_{t-1})$

Ensure: Executed control variables (ω_t, v_t) and the updated kinematic state

1: Map $u_{\theta,t}$ and $u_{v,t}$ to the unconstrained motion commands $(\omega_t^{pre}, v_t^{pre})$
2: Map $u_{L,t}$ to the look-ahead distance L_t
3: Compute the unconstrained linear acceleration a_t^{pre} from v_t^{pre} and v_{t-1}
4: Compute the unconstrained linear jerk j_t^{pre} from a_t^{pre} and a_{t-1}
5: Clip j_t^{pre} to obtain j_t under the prescribed jerk bound
6: Recover the linear acceleration a_t from j_t and apply acceleration clipping
7: Integrate (v_{t-1}, a_{t-1}, j_t) to obtain the linear velocity v_t for the current control cycle
8: Apply the same procedure to the angular-velocity channel to obtain the constrained angular velocity ω_t
9: Return the executed control variables (ω_t, v_t) and the updated kinematic state

The intervention degree of the KCM is defined as

$$\eta_t = \frac{|v_t - v_t^{pre}|}{V_{\max}} + \frac{|\omega_t - \omega_t^{pre}|}{\Omega_{\max}}. \tag{20}$$

This scalar is not part of the primary optimization objective. It is used only as an auxiliary metric in the result analysis.

3.4. Reward Design

The instantaneous reward is

$$r_t = r_t^{prog} + r_t^{track} + r_t^{dir} + r_t^{smooth}. \tag{21}$$

The progress reward is defined as

$$r_t^{prog} = w_p \max(0, \rho_t - \rho_{t-1}), \tag{22}$$

where ρ_t is the overall path progress.

The contour-error penalty is defined as

$$\phi_e(e_t) = \begin{cases} 0, & |e_{n,t}| \le \delta_t, \\ \left(\dfrac{|e_{n,t}| - \delta_t}{\varepsilon/2} \right)^2, & |e_{n,t}| > \delta_t, \end{cases} \tag{23}$$

and the corresponding tracking reward is

$$r_t^{track} = -w_e(c_t)\phi_e(e_t). \tag{24}$$

Here, δ_t is the relaxed boundary within the tolerance band, and $c_t \in [0, 1]$ is the local corner intensity. They are defined as

$$\delta_t = \kappa_\delta \frac{\varepsilon}{2} \mathbf{1}(m_t = 1), \tag{25}$$

$$c_t = \mathrm{clip}\left((1 - \alpha_c) c_{t-1} + \alpha_c \frac{|\Delta\theta_t^{(N_L)}|}{\theta_0}, 0, 1 \right), \tag{26}$$

where κ_δ is the error-relaxation ratio, m_t indicates whether the current state is within the corner-influence region, $\Delta\theta_t^{(N_L)}$ is the look-ahead directional change at the farthest scale, θ_0 is the normalization angle for local corner intensity, and $\alpha_c = 1/T_c$ is the exponential moving-average coefficient. In the main experiments, κ_δ is set to 0. Thus, the contour-error term has no additional dead zone, while c_t is still used to adjust the tracking and smoothness weights.

The directional consistency term is defined as

$$r_t^{dir} = -w_\tau |\tau_t|, \tag{27}$$

where τ_t is the angular error between the current heading and the local reference direction.

The smoothness term is defined as

$$r_t^{smooth} = -w_s(c_t)\psi_t, \tag{28}$$

where ψ_t can be a normalized measure of linear jerk, angular acceleration, or angular jerk. The weight $w_s(c_t)$ is adjusted according to the local corner intensity. One implementation is

$$w_s(c_t) = w_s^0 c_t^\gamma, \qquad \gamma > 1, \tag{29}$$

where $w_s(c_t)$ increases with c_t.

3.5. PPO Training Configuration

The policy is trained with Proximal Policy Optimization (PPO). The Actor is a shared multilayer perceptron with three hidden layers of widths 256, 128, and 64. Each layer uses ELU activation followed by LayerNorm. Separate output heads parameterize the mean and standard deviation of a three-dimensional action distribution. The steering mean is mapped to $[-1, 1]$ with tanh, while the feedrate and look-ahead means are mapped to $[0, 1]$ with the sigmoid function. The standard deviation is obtained by applying the softplus function and then adding 10^{-3} for numerical stability. The Critic has layer widths $256, 128, 64, 1$ and uses ELU activation, LayerNorm, and a dropout rate of 0.1. The main training hyperparameters are listed in Tab. 1.

Tab. 1: PPO hyperparameters and training protocol.

Parameter	**Value**
Actor learning rate	3.0×10^{-4}
Critic learning rate	1.5×10^{-4}
Discount factor γ	0.99
GAE parameter λ	0.95
PPO clip coefficient	0.1
Number of epochs per update	8
Entropy coefficient	0.01
Gradient clipping	0.5

4. Experiments and Results

This section evaluates the motion-planning performance of the proposed method on geometric paths with different characteristics through simulation experiments. The evaluation focuses on three aspects: whether the tool motion can advance stably along the reference path, whether the planned trajectory can generate reasonable transitions within the tolerance band, and whether the executed control variables satisfy the prescribed velocity, acceleration, and jerk constraints. The evaluation consists of a main comparative experiment and a set of ablation experiments. The main comparative experiment compares J-NNC with the neural network controller (NNC) baseline, whereas the ablation experiments separately modify the preview distance, KCM, preview observation, and reward scheduling to analyze the contribution of each module to the overall performance.

4.1. Experimental Settings

The experiments use three representative paths: a square path, a circular path, and a butterfly-shaped path. The circular path has continuous turning characteristics and is mainly used to examine the ability of the policy to maintain the feedrate and control the contour error under continuous-curvature conditions. The square path contains typical sharp corners and is used to evaluate the deceleration, turning, and re-acceleration behavior of the policy during corner entry, corner traversal, and corner exit. The butterfly-shaped path includes local curvature variations and geometric shape changes, and is used to assess the continuous path-following capability and trajectory control performance of the policy on complex geometric paths.

The main comparative experiment compares J-NNC with the NNC baseline. J-NNC adopts a structured state representation, a learnable look-ahead distance, KCM-based

constraint enforcement, and a corner-aware reward. The NNC baseline is constructed following the direct neural control framework proposed by Li et al. [2], in which the policy output is directly applied to the environment for execution. The comparison between the two methods focuses on path advancement capability, contour error, and satisfaction of kinematic constraints.

The ablation experiments are constructed by modifying the proposed method. In the fixed-look-ahead configuration, the learnable look-ahead distance is replaced with a fixed look-ahead length. In the no-KCM configuration, the execution-layer kinematic-constraint projection is removed, so that the policy output is directly executed in the environment. In the no-look-ahead-observation configuration, the multi-scale forward geometric input is removed. In the no-straight/corner-dual-reward configuration, the differentiated reward scheduling for straight and corner segments is disabled. These configurations allow the effects of look-ahead distance, execution constraints, forward geometric information, and reward structure on the experimental results to be examined separately.

For evaluation, the best rollout exported from each configuration is used as the statistical sample instead of an average over multiple evaluation episodes. The metrics include the terminal state, final progress, maximum contour error, maximum relative violation of the linear-jerk limit, and termination time. The terminal state and final progress characterize path advancement, the maximum contour error measures the deviation of the trajectory from the reference path, and the maximum relative violation of the linear-jerk limit indicates whether the executed control commands satisfy the jerk constraint. The termination time is obtained by multiplying the terminal step of the best rollout by the interpolation period. Since this work studies constrained motion planning, the subsequent analysis jointly considers efficiency, error, and dynamic feasibility.

4.2. Main Comparative Results

This subsection reports the comparative results of J-NNC and the NNC baseline on three representative paths. Tab. 2 summarizes the termination status, final progress, maximum contour error, maximum relative violation of the linear-jerk limit, and termination time for each method.

Tab. 2: Comparison between J-NNC and the NNC baseline on representative best rollouts.

Path	**Metric**	**J-NNC**	**NNC baseline**
square	Termination status	success	success
	Final progress	1.000	1.000
	Maximum contour error	0.834	0.676
	Maximum relative linear-jerk exceedance	0.000	3199.000
	Termination time (s)	12.192	8.672
circle	Termination status	success	max_steps
	Final progress	1.000	0.990
	Maximum contour error	0.096	6.259
	Maximum relative linear-jerk exceedance	0.000	3199.000
	Termination time (s)	9.117	60.000
butterfly	Termination status	success	success
	Final progress	1.000	1.000
	Maximum contour error	0.745	0.207
	Maximum relative linear-jerk exceedance	0.000	3199.000
	Termination time (s)	10.480	5.807

Tab. 2 shows that J-NNC achieves successful termination on the square, circular, and butterfly-shaped paths. The NNC baseline also terminates successfully on the square and butterfly-shaped paths, whereas it reaches the maximum step limit on the circular path. A further comparison indicates that the main advantage of J-NNC lies in its ability to satisfy the kinematic constraints. For all three paths, J-NNC keeps the maximum relative violation of linear jerk at zero. The NNC baseline demonstrates path-advancement capability, but without an explicit execution-layer projection, its raw actions may violate jerk limits under the tested constraints. After KCM is introduced, the policy outputs are converted into executable control commands that satisfy the velocity, acceleration, and jerk limits.

From the perspective of path geometry, the circular path mainly evaluates control performance under continuous turning. The results for the circular path in Tab. 2 show that J-NNC terminates successfully and keeps both the contour error and the jerk constraint within acceptable ranges. The NNC baseline reaches the maximum step limit on this path. The square path contains sharp corners and a closed endpoint. Both methods complete this path, and their differences mainly appear in trajectory handling near the corners and in jerk-constraint satisfaction. The trajectory generated by the NNC baseline is closer to the original polyline, with more concentrated direction changes. J-NNC forms local transitions within the tolerance band and limits the executed control commands through KCM. The butterfly-shaped path involves more complex curvature variations. The NNC baseline may show numerical advantages in some efficiency or error metrics, yet the absence of explicit jerk-constrained projection limits its dynamic executability under the prescribed machine-tool constraints. In contrast, J-NNC satisfies the kinematic constraints through a more conservative execution process.

Fig. 4 presents the trajectory comparisons on the three paths.

In Fig. 4(a)–(f), the trajectories generated by J-NNC remain within the tolerance band and do not exactly coincide with the reference polyline. On the square path, Fig. 4(a) shows that J-NNC forms local corner transitions, whereas Fig. 4(b) shows that the NNC baseline follows the original polyline more closely. On the circular path,

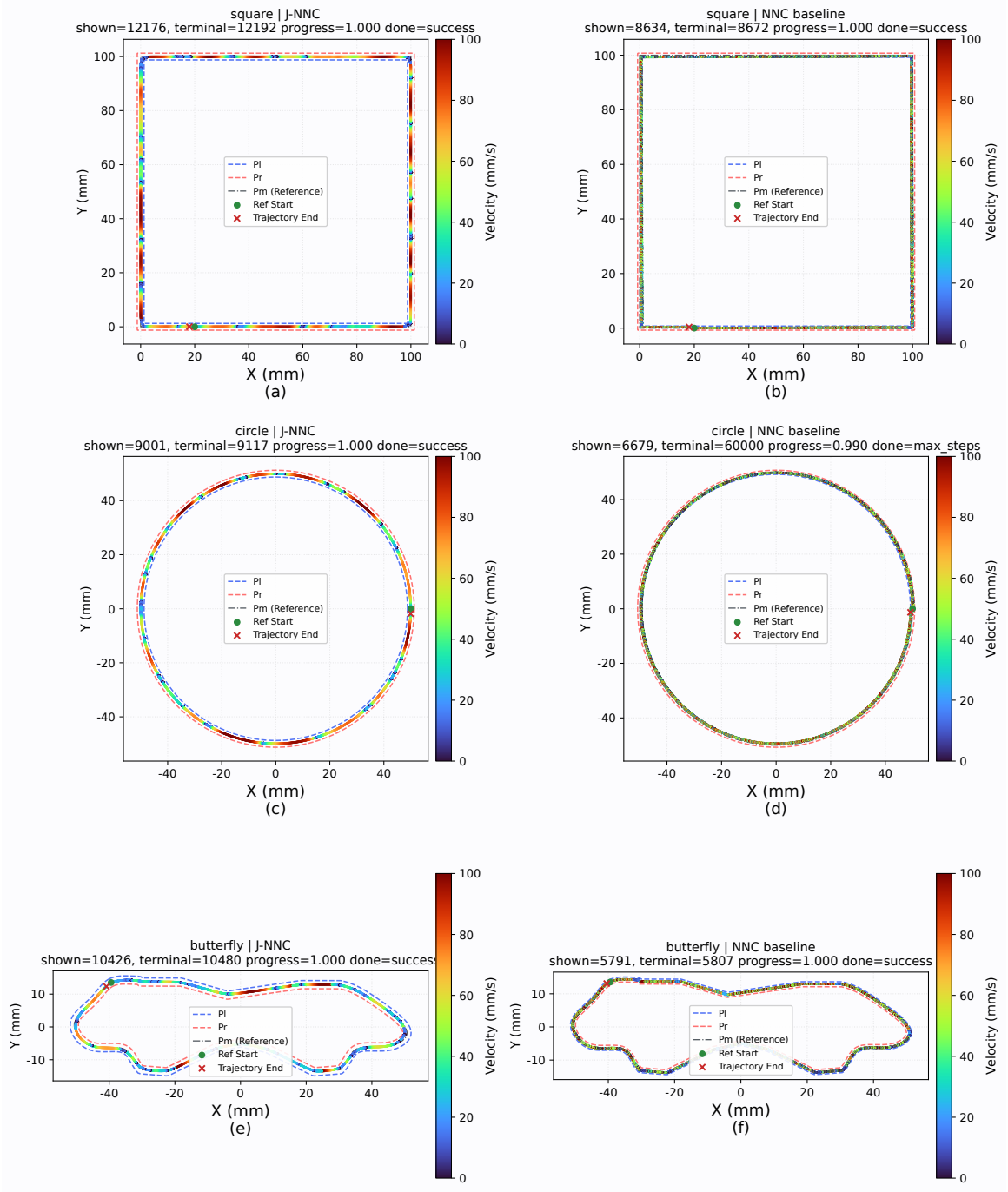

Fig. 4: Trajectory comparison between J-NNC and the NNC baseline on representative paths: (a) square path, J-NNC; (b) square path, NNC baseline; (c) circular path, J-NNC; (d) circular path, NNC baseline; (e) butterfly-shaped path, J-NNC; (f) butterfly-shaped path, NNC baseline.

Fig. 4(c) shows continuous advancement with relatively smooth velocity variation, while Fig. 4(d) reflects the baseline behavior under continuous turning. On the butterfly-shaped path, Fig. 4(e) shows moderate offsets in curvature-changing regions, while Fig. 4(f) provides the corresponding NNC baseline. These observations indicate that J-NNC uses the tolerance band to adjust the local trajectory, thereby reducing abrupt control changes near corners or varying-curvature regions.

The corner region of the square path is compared in Fig. 5.

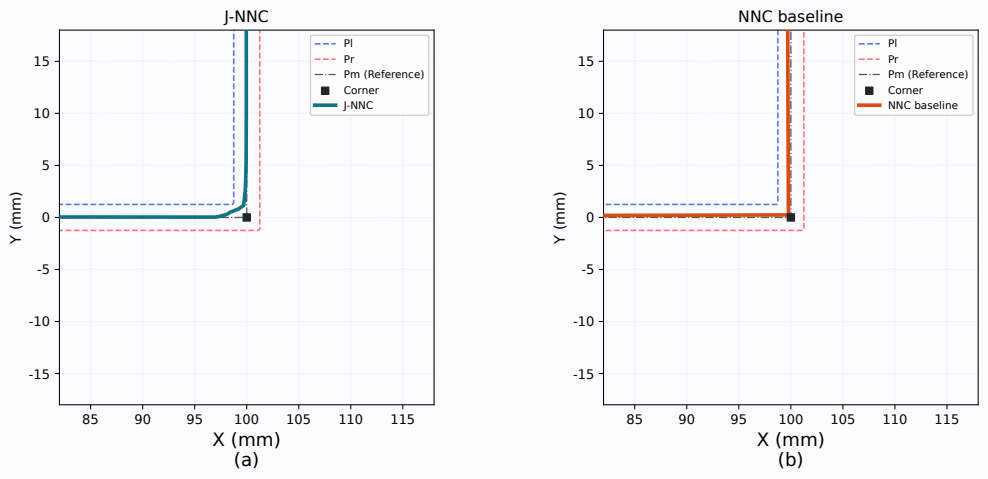

Fig. 5: Local zoomed-in comparison in the square-corner region: (a) J-NNC; (b) NNC baseline.

Fig. 5(a) shows that J-NNC begins to change the motion direction before approaching the corner and forms a continuous transition trajectory within the tolerance band. In contrast, Fig. 5(b) shows that the NNC baseline remains closer to the original polyline, with the direction change concentrated near the corner point. This observation is consistent with the jerk results reported in Tab. 2. When the policy outputs are executed directly, large dynamic variations tend to occur at the corners. After the KCM constraint is applied, J-NNC satisfies the jerk limit during turning.

The normalized time histories of contour error, velocity, and linear jerk for J-NNC on a representative best rollout are shown in Fig. 6.

In Fig. 6(a), the peak contour errors mainly occur during cornering, indicating that the policy uses the available spatial margin within the tolerance band. Fig. 6(b) reflects the speed modulation process during corner entry, corner traversal, and corner exit. Fig. 6(c) reports the windowed peak $|j|/J_{\max}$ curve obtained by rechecking the velocity records of the same best rollout against the KCM boundary. This curve remains no greater than 1. Together with Tab. 2, this result shows that the maximum relative exceedance of linear jerk is zero after KCM is enabled.

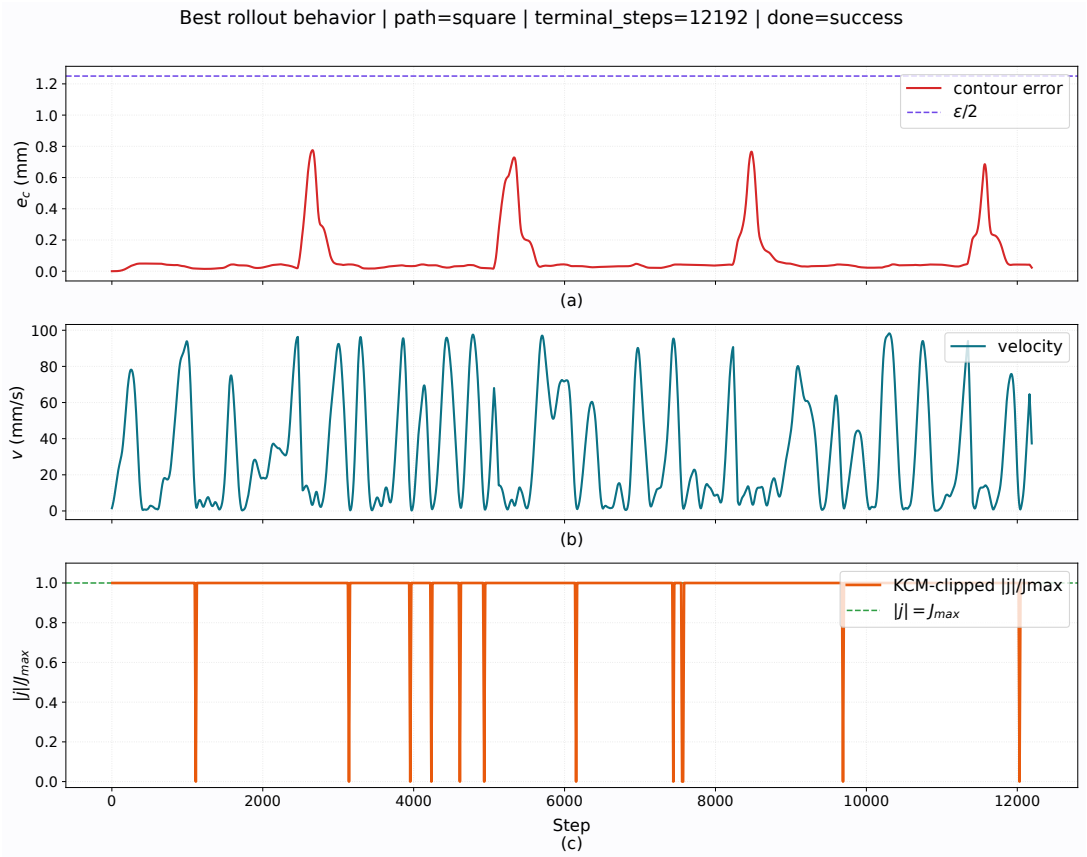

Fig. 6: Representative best-rollout behavior of J-NNC: (a) contour error; (b) velocity; (c) normalized linear jerk.

The combined results in Tab. 2 and Fig. 4–Fig. 6 show that J-NNC completes the best rollout on all three types of paths and forms locally smooth trajectories within the tolerance band in both high-speed linear feeding segments and corner regions. For the sharp corners of the square path and the mixed-curvature regions of the butterfly-shaped path, J-NNC keeps the executed control quantities within the kinematic boundaries at corners or curvature-transition regions through appropriate trajectory smoothing and KCM constraints. The NNC baseline demonstrates path-advancement capability, while the absence of explicit constraint handling limits its dynamic feasibility.

4.3. Ablation Results

The ablation study removes or modifies the learnable look-ahead distance, KCM, look-ahead observation, and dual straight-line/corner reward modules. Tab. 3 reports, for the best rollout on each path, the termination status, final progress, maximum contour error, maximum relative violation of the linear jerk limit, and termination time under each configuration.

Tab. 3: Path-level best-rollout results of the structured ablation study.

Model configuration	Path	Termination status	Final progress	Maximum contour error	Maximum relative linear-jerk violation	Termination time (s)
J-NNC	square	success	1.000	0.834	0.000	12.192
J-NNC	circle	success	1.000	0.096	0.000	9.117
J-NNC	butterfly	success	1.000	0.745	0.000	10.480
Fixed look-ahead	square	success	1.000	0.699	0.000	8.680
Fixed look-ahead	circle	success	1.000	0.046	0.000	4.804
Fixed look-ahead	butterfly	success	1.000	0.638	0.000	5.049
Without KCM	square	success	1.000	0.722	3199.000	10.301
Without KCM	circle	success	1.000	0.061	3199.000	5.926
Without KCM	butterfly	success	1.000	0.707	3199.000	7.446
Without look-ahead observation	square	success	1.000	0.805	0.000	8.151
Without look-ahead observation	circle	success	1.000	0.077	0.000	4.573
Without look-ahead observation	butterfly	success	1.000	0.820	0.000	6.044
Without straight-line/corner dual rewards	square	success	1.000	0.824	0.000	15.132
Without straight-line/corner dual rewards	circle	success	1.000	0.030	0.000	10.388
Without straight-line/corner dual rewards	butterfly	success	1.000	0.918	0.000	9.623

Tab. 3 shows that all ablated configurations terminate successfully in the best rollouts on the three paths, although they differ in termination time, contour error, and jerk constraint satisfaction. The fixed look-ahead setting isolates the effect of online look-ahead distance adjustment. Removing look-ahead observation tests the contribution of multi-scale geometric information, while removing the dual straight-line/corner reward tests the effect of phase-dependent reward weights. The setting without KCM is used to examine the role of constraint projection at the execution layer. Since KCM is enabled in all other settings, the maximum relative violation of the linear jerk limit is zero for those settings.

Fig. 7 compares the linear jerk of J-NNC with that of the setting without KCM on the same representative path.

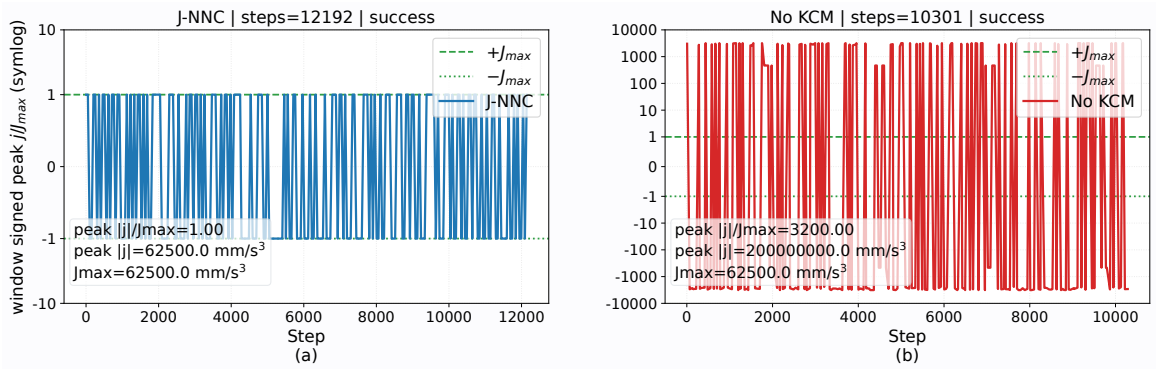

Fig. 7: Linear-jerk constraint comparison on the same representative path: (a) J-NNC with KCM; (b) ablated configuration without KCM.

As shown in Fig. 7(a), the linear jerk of J-NNC stays within the bound defined by $J_{\max}$. By contrast, Fig. 7(b) shows high-amplitude jerk spikes after KCM is removed. This result suggests that shorter execution time or smaller contour error alone is not sufficient to guarantee an executable trajectory. KCM is therefore a necessary part of the proposed method because it maps policy outputs to executable control commands that satisfy velocity, acceleration, and jerk constraints.

The fixed look-ahead setting examines the role of look-ahead distance adjustment. It retains the forward geometric perception structure but uses a fixed look-ahead length for decision making. As shown in Tab. 3, this setting terminates successfully on all three paths, and it produces shorter termination times on some paths. The learnable look-ahead distance allows the policy to adjust the perception scale online, while also increasing the dimensionality of the action space. Its effect depends on path complexity, reward weights, and the degree of KCM intervention. The current results suggest that

the parameterization of the look-ahead distance action still needs refinement, so that it can support anticipatory speed regulation and corner preprocessing more consistently. For this reason, its role should be assessed with metrics beyond termination time alone.

The setting without look-ahead observation examines the role of multi-scale forward geometric information. It retains the learnable look-ahead distance and KCM-constrained execution, but removes the multi-scale look-ahead observation state. As shown in Tab. 3, the local state and corner awareness are sufficient for basic progression on all three paths, with path-dependent changes in termination time and contour error. Multi-scale look-ahead observations give the policy more information about the upcoming geometric trend. This information is mainly useful for anticipatory deceleration, early steering, and velocity transition on complex hybrid paths. The benefit of this module should be further tested on longer paths, denser corner distributions, and more complex curvature profiles.

The setting without the straight-line/corner dual-reward scheme examines the role of phase-dependent reward design. It applies the same reward weights to straight-line feeding, corner entry, corner traversal, and corner exit. As shown in Tab. 3, removing this module reduces completion stability across paths. Straight segments mainly require stable feed motion and directional consistency, whereas corner segments require error regulation, limited velocity variation, and smooth steering. When the same reward weights are used for all phases, the policy is more prone to unstable behavior during transitions between straight segments and corner regions. The straight-line/corner dual-reward scheme therefore helps the policy learn control behaviors that depend on the local path geometry.

The results in Tab. 3 and Fig. 7 indicate that KCM mainly ensures the dynamic feasibility of the executed trajectory, while the straight-line/corner dual-reward scheme mainly improves behavioral stability across geometric phases. The learnable look-ahead distance and multi-scale look-ahead observations provide forward geometric perception and online adjustment capability. The current results also show that, although the square closed path can be completed successfully, termination time and velocity continuity after corner traversal remain important factors for experimental efficiency and should be addressed in subsequent optimization.

5. Conclusion

This paper presented J-NNC, a reinforcement learning-based CNC motion-planning method for mixed CAM-generated tool paths subject to contour-tolerance and kinematic constraints. The main difficulty considered in this work is that geometric smoothing, feedrate planning, and dynamic feasibility cannot be treated independently in practical CNC interpolation. A trajectory that stays close to the reference polyline may still contain sharp direction changes near corners, while a learned controller that advances the path efficiently may produce commands that exceed velocity, acceleration, or jerk limits. J-NNC addresses this issue by treating CNC interpolation as a closed-loop constrained decision-making process, rather than as a serial procedure with separate smoothing and feedrate-planning stages. The structured state and learnable preview action allow the policy to combine tracking, kinematic, corner, and forward-geometric information when planning local transitions within the tolerance band. The KCM further projects learned steering and feedrate outputs into executable commands satisfying velocity, acceleration, and jerk limits. Experiments on square, circular, and butterfly-shaped paths show full path progress and zero maximum relative linear-jerk violation, confirming the importance of explicit action projection compared with the NNC baseline and the no-KCM ablation.

Future work will refine the preview-distance action and corner-phase representation to improve traversal efficiency and velocity continuity after corner transitions. Further validation will include servo lag, axis-level constraints, and real machining experiments to assess practical deployment on CNC systems.

References

[1] Yuwen Sun, Jinjie Jia, Jinting Xu, Mansen Chen, and Jinbo Niu. Path, feedrate and trajectory planning for free-form surface machining: A state-of-the-art review. *Chinese Journal of Aeronautics*, 35(8):12–29, 2022.

[2] Bingran Li, Hui Zhang, Peiqing Ye, and Jinsong Wang. Trajectory smoothing method using reinforcement learning for computer numerical control machine tools. *Robotics and Computer-Integrated Manufacturing*, 61:101847, 2020.

[3] Guangwen Yan, Desheng Zhang, Jinting Xu, and Yuwen Sun. Corner smoothing for CNC machining of linear tool path: A review. *Journal of Advanced Manufacturing Science and Technology*, 3(2):2023001, 2023.

[4] Bingcai Wu, Junyan Ma, Lutao Wei, Xiaoping Liao, and Juan Lu. NURBS interpolator with scheduling scheme combining cubic and quartic S-shaped feedrate profiles under drive and chord error constraints. *Computer-Aided Design*, 152:103380, 2022.

[5] Yifei Hu, Xin Jiang, Guanying Huo, Cheng Su, Hexiong Li, Li-Yong Shen, and Zhiming Zheng. Enhancing five-axis CNC toolpath smoothing: Overlap elimination with asymmetrical B-splines. *CIRP Journal of Manufacturing Science and Technology*, 52:36–57, 2024.

[6] Hexiong Li, Xin Jiang, Guanying Huo, Cheng Su, Shiwei Zhou, Bolun Wang, Yifei Hu, and Zhiming Zheng. An integrated trajectory smoothing method for lines and arcs mixed toolpath based on motion overlapping strategy. *Journal of Manufacturing Processes*, 95:242–265, 2023.

[7] Huan Zhao, Limin Zhu, and Han Ding. A real-time look-ahead interpolation methodology with curvature-continuous B-spline transition scheme for CNC machining of short line segments. *International Journal of Machine Tools and Manufacture*, 65:88–98, 2013.

[8] De-Ning Song, Jian-Wei Ma, Yu-Guang Zhong, and Jian-Jun Yao. Global smoothing of short line segment toolpaths by control-point-assigning-based geometric smoothing and FIR filtering-based motion smoothing. *Mechanical Systems and Signal Processing*, 160:107908, 2021.

[9] S. Sun, P. Zhao, T. Zhang, B. Li, and D. Yu. Smoothing interpolation of five-axis tool path with less feedrate fluctuation and higher computation efficiency. *Journal of Manufacturing Processes*, 109:669–693, 2024.

[10] A. C. Lee, M. T. Lin, Y. R. Pan, and W. Y. Lin. The feedrate scheduling of NURBS interpolator for CNC machine tools. *Computer-Aided Design*, 43(6):612–628, 2011.

[11] Kaan Erkorkmaz and Yusuf Altintas. High speed CNC system design. part I: jerk limited trajectory generation and quintic spline interpolation. *International Journal of Machine Tools and Manufacture*, 41(9):1323–1345, 2001.

[12] De-Ning Song, Yu-Guang Zhong, and Jian-Wei Ma. Look-ahead-window-based interval adaptive feedrate scheduling for long five-axis spline toolpaths under axial drive constraints. *Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture*, 234(13):1656–1670, 2020.

[13] Yong Zhang, Hui Zhang, Jiadong Shi, Jiali Jiang, and Mingyong Zhao. Development of an adaptive-feedrate planning and iterative interpolator for parametric toolpath with normal jerk constraint. *International Journal of Precision Engineering and Manufacturing*, 22:73–81, 2021.

[14] Haiming Zhang, Jianzhong Yang, Wanqiang Zhu, Chenglei Zhang, and Zifang Hu. Jerk-constrained feedrate scheduling for CNC machining based on constraint-guided sampling. *The International Journal of Advanced Manufacturing Technology*, 2026.

[15] Yunan Wang, Chuxiong Hu, Jichuan Yu, Shize Lin, Zhao Jin, and Jizhou Yan. Jerk-limited oscillation-free feedrate scheduling under non-stationary boundary conditions. In *2025 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*, pages 1–6, 2025.

[16] Mingxi Zhong, Yanyang Liang, Wenxuan Xie, Wei Cui, Hongfei Lv, and Dongzhou Zhong. Real-time dynamic look-ahead trajectory planning for industrial robots based on visual feedback. *The International Journal of Advanced Manufacturing Technology*, 140:6045–6063, 2025.

[17] Vladimir Samsonov, Chrismarie Enslin, Hans-Georg Köpken, Schirin Bär, Daniel Lütticke, and Tobias Meisen. Deep representation learning and reinforcement learning for workpiece setup optimization in CNC milling. *Production Engineering*, 17(6):847–859, 2023.

[18] Seyed Adel Alizadeh Kolagar, Mehdi Heydari Shahna, and Jouni Mattila. Combining deep reinforcement learning with a jerk-bounded trajectory generator for kinematically constrained motion planning. In *2025 European Control Conference (ECC)*, pages 2507–2514. IEEE, 2025.

[19] Tu-Hoa Pham, Giovanni De Magistris, and Ryuki Tachibana. OptLayer – practical constrained optimization for deep reinforcement learning in the real world. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6236–6243. IEEE, 2018.

[20] Mohammed Alshiekh, Roderick Bloem, Rüdiger Ehlers, Bettina Könighofer, Scott Niekum, and Ufuk Topcu. Safe reinforcement learning via shielding. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1):2669–2678, 2018.

[21] Richard Cheng, Gábor Orosz, Richard M. Murray, and Joel W. Burdick. End-to-end safe reinforcement learning through barrier functions for safety-critical continuous control tasks. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33(01):3387–3395, 2019.

[22] Kim Peter Wabersich and Melanie N. Zeilinger. A predictive safety filter for learning-based control of constrained nonlinear dynamical systems. *Automatica*, 129:109597, 2021.